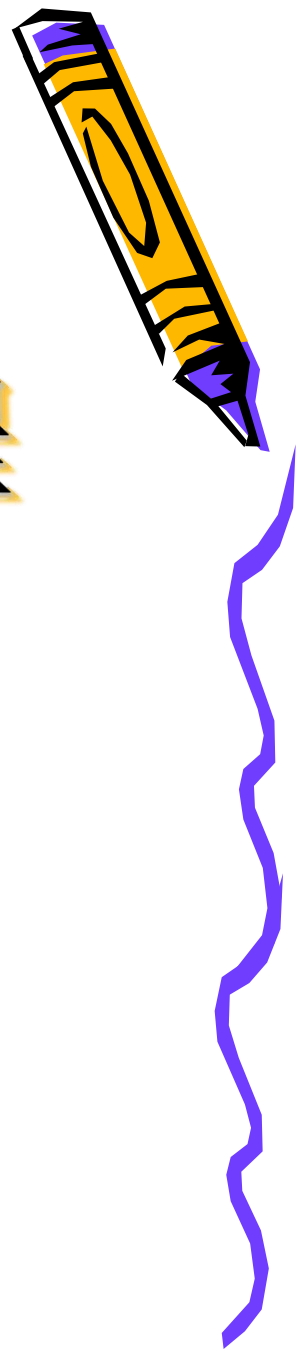


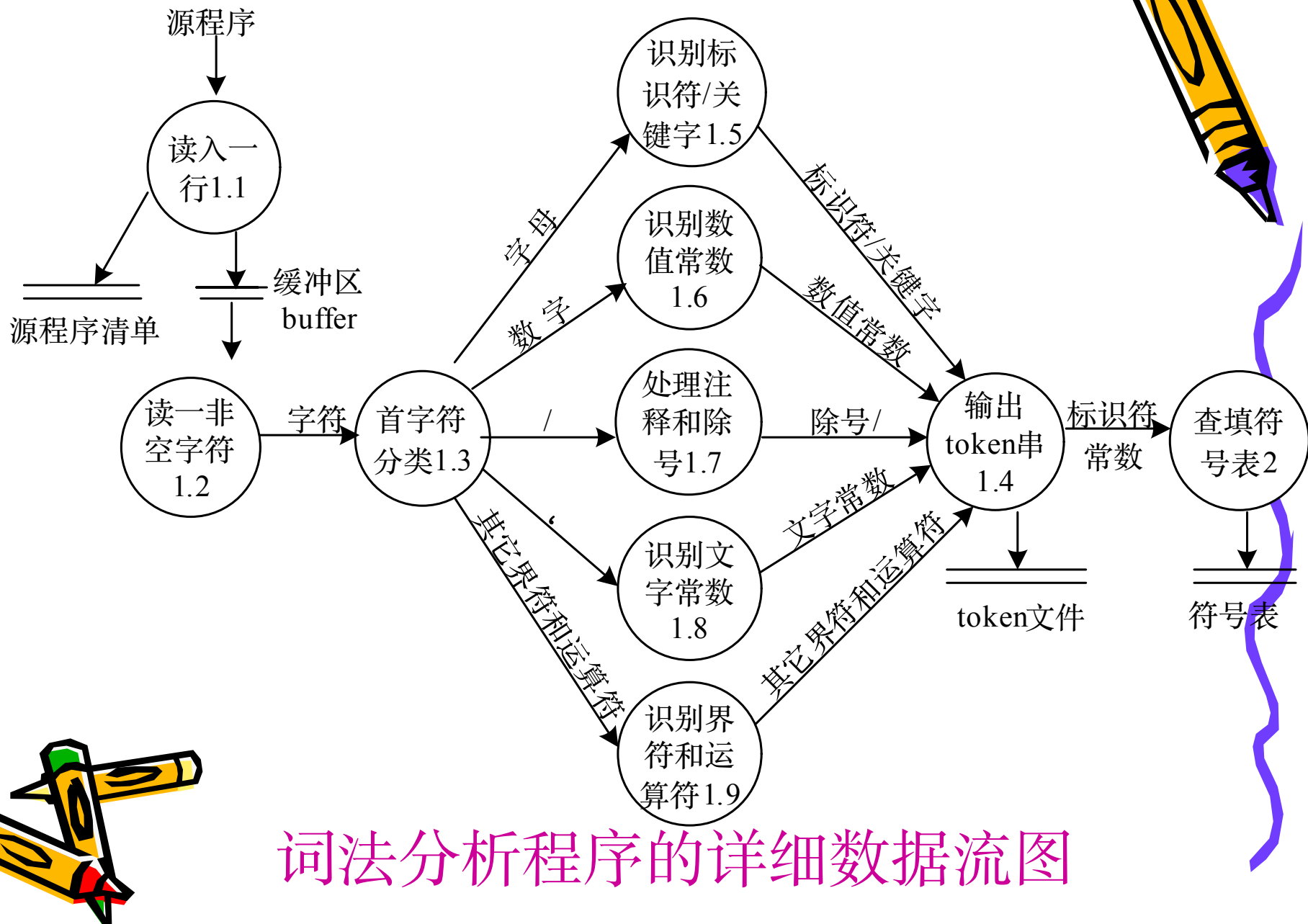
编译原理及实践教程

河南师范大学
软件学院

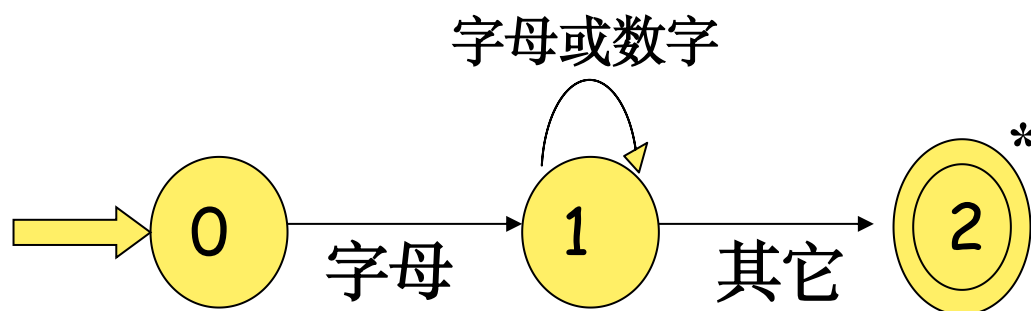
张聪品



回顾



回顾：状态转换图



识别标识符的转换图

识别过程是：从初始状态0开始，若读入一个字母，转入1状态，若再读入字母或数字，仍处于1状态，否则转向2状态，结束一个标识符的识别过程。状态上的*表示多读入一个符号。



写成类C语言的函数形式：

```
recog_id()
```

```
{ ... char state = 0;
```

```
  ch = getch();
```

```
  do {
```

```
    switch(state){
```

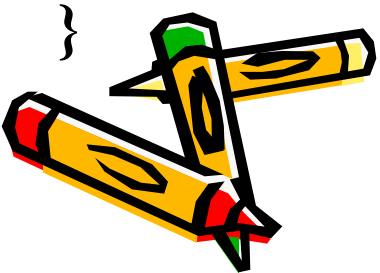
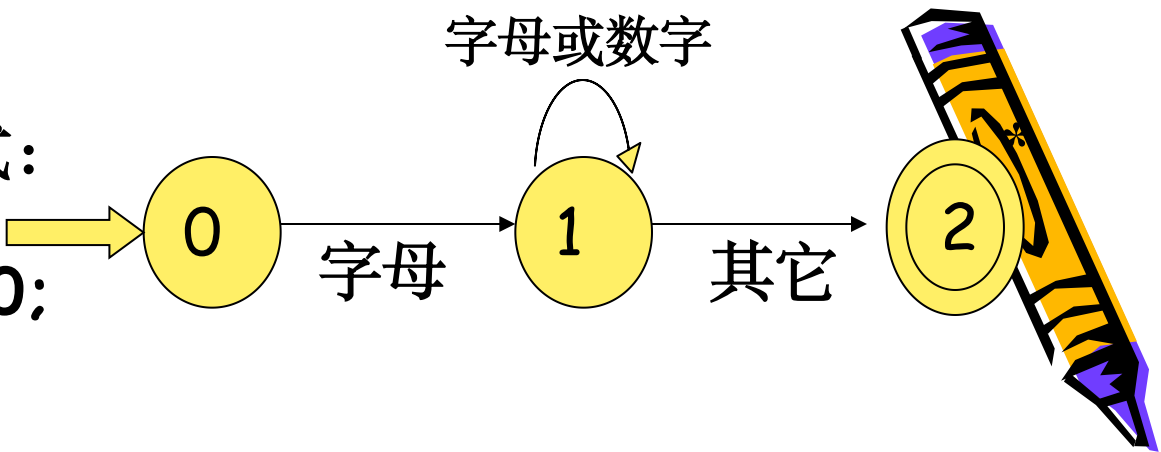
```
      Case 0: if ch 是字母 state = 1; ch =  
getch();break;
```

```
      Case 1: if ch 是字母或数字 {  
                state = 1; ch = getch();}  
                else state = 2;  
                break;
```

```
    }
```

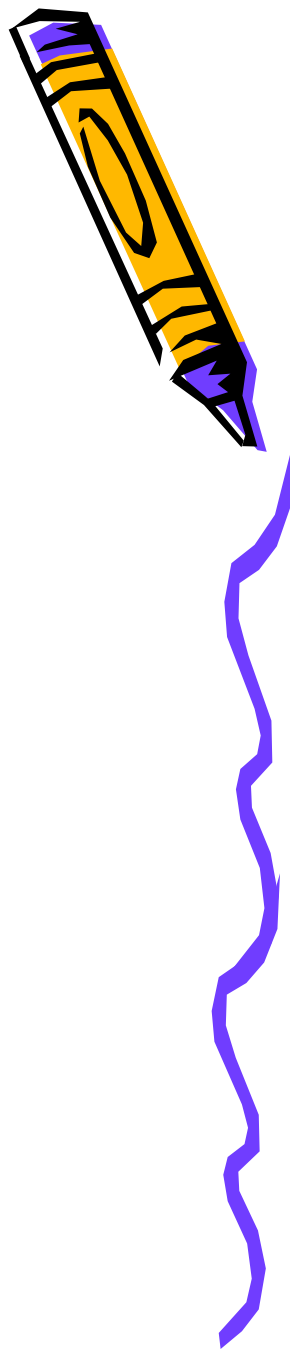
```
  } while (state != 2);  
  return( gettoken() )
```

```
}
```



词法分析器的自动生成

- 正规式
- 有穷自动机



2.5 正规式与有穷自动机

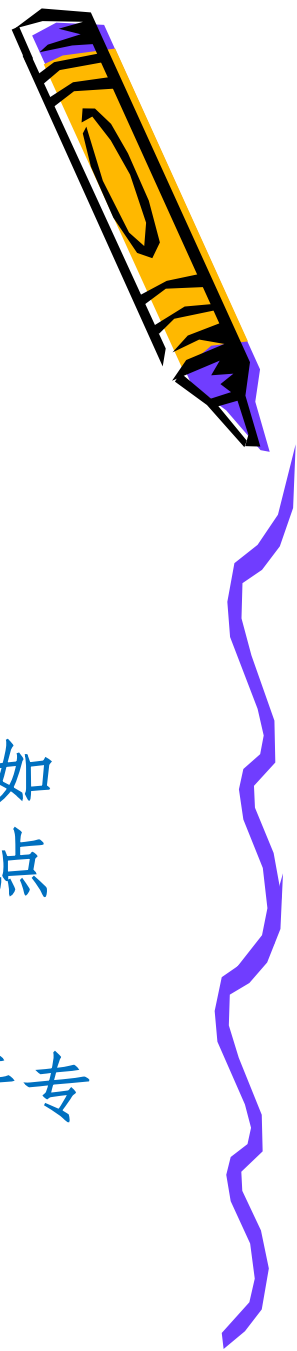


- 1 能根据自然语言描述构造正规式、NFA
- 2 掌握NFA转换为DFA，DFA的化简
- 3 掌握正规式和有穷自动机间的转换



2.5.1

符号和符号串



1. **字母表**：高级语言程序能够使用的全体字符构成的集合，是元素的有穷非空集合 Σ

不同的语言可以有不同的字母表，例如英文的字母表中26个字母、数字及标点符号等。

C语言的字母表是由字母、数字、若干专用符号组成。



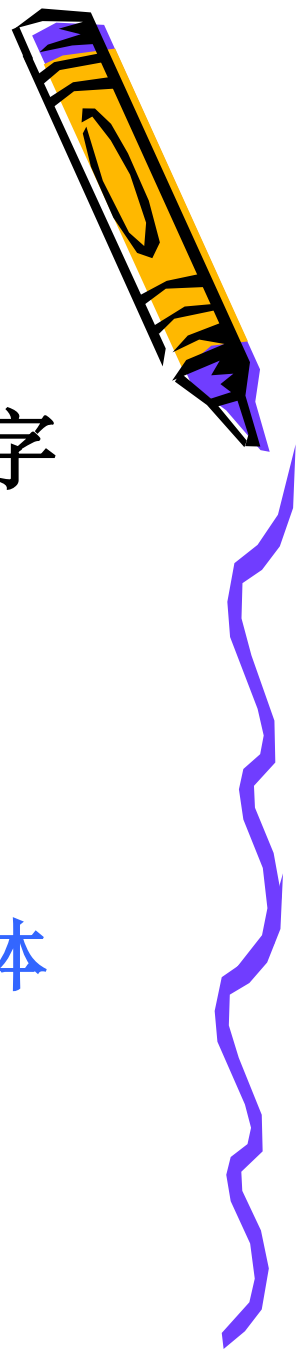
2.5.1

符号和符号串

2. **符号**：字母表中的每个元素，因此字母表也称为符号集。

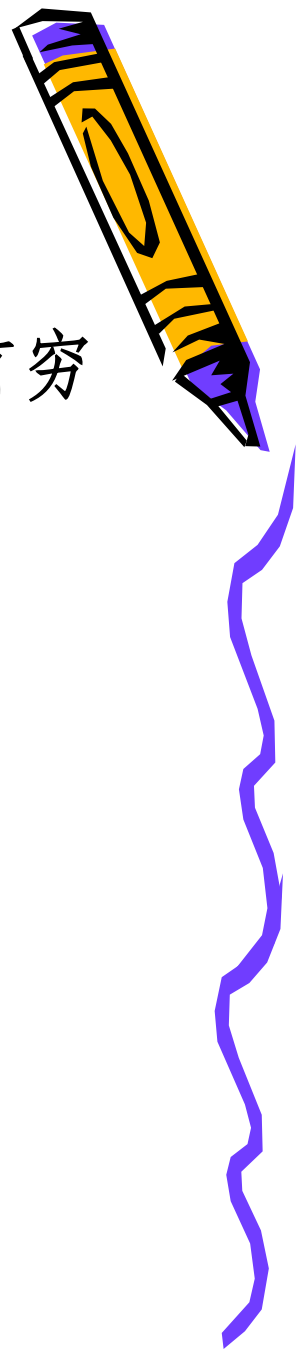
符号是某语言能识别的字符。

字母表是该语言能识别的所有符号的全体（字符集）。



2.5.1

符号和符号串



3. **符号串**：由字母表中的符号组成的任何有穷序列称为符号串。

- 如001110 是字母表 $\Sigma = \{0, 1\}$ 上的符号串

字母表 $A = \{a, b, c\}$ 上的一些符号串有：

a, b, c, ab, aaca等。

C语言中的单词、语句、程序是符号串吗？

C语言字母表上的符号串是程序吗？



符号串的长度 : 符号串中符号的个数

如：某符号串中有 m 个符号, 则称其长度为 m , 表示为 $|x|=m$, 如001110的长度是6。

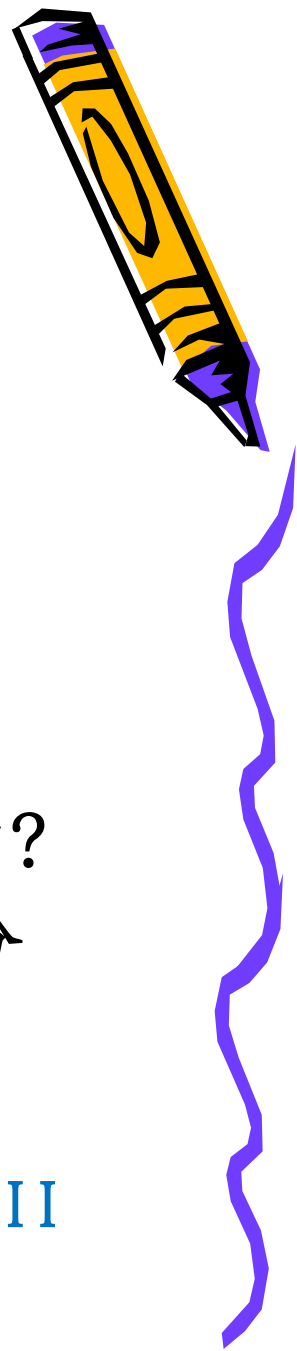
空符号串：即不包含任何符号的符号串，用 ε 表示，其长度为0， 即 $|\varepsilon|=0$ 。

在符号串中，符号的顺序是很重要的，符号串 ab 就不同于 ba ， $abca$ 和 $aabc$ 也不同。符号串 STR 表示“由符号 S 、 T 和 R ，并按此顺序组成



2.5.1

符号和符号串



• 4. 符号串的连接运算

符号串的**连接**：字符串 α β 称为字符串 α 和 β 的连接

如： $\alpha=01$ ， $\beta=110$ ， 则 $\alpha\beta=01110$ ，
 $\beta\alpha=11001$

C语言的字母表中的字母如何构成单词的？
单词是如何构成语句的？ 语句等语法成分
是如何构成程序的？



所有语法正确的C语言程序的集合，是ASCII
字符集上的符号串的集合

2.5.2 集合的运算及语言定义



1 集合运算



(1) 集合的并：

$$U \cup V = \{ \alpha \mid \alpha \in U \text{ 或者 } \alpha \in V \}$$



(2) 集合U、V的连接：

$$UV = \{ \alpha \beta \mid \alpha \in U \text{ \& } \beta \in V \}$$

即：集合UV中的符号串是由U和V的符号串连接而成。

$U = \{aa, bb\}$ $V = \{00, 11\}$ 则 $UV = ?$

集合UV中的符号串是由U和V的符号串连接而成。



2.5.2

集合的运算及语言定义

✚ (3) 集合的**方幂**：n个相同集合的连接

$$V^n = VVV \dots V \quad \text{规定 } V^0 = \{\varepsilon\}$$

(4) 集合的闭包、正闭包

Σ 的闭包 Σ^* 表示 Σ 上的所有符号串（包括 ε ）组成的集合。 长度为 0, 1, 2, ...

Σ 的正闭包 Σ^+ 表示 Σ^* 上的除 ε 外的所有符号串组成的集合。



$$\Sigma^* = \{\varepsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

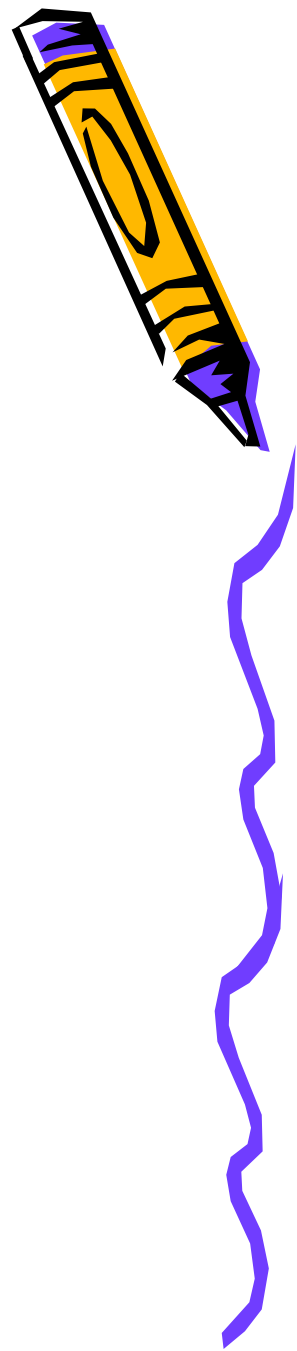
$$\Sigma^+ = \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

例: $\Sigma = \{a, b\}$

$\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$

$\Sigma^+ = \{a, b, aa, ab, ba, bb, aaa, aab, \dots\}$

$V = \{0, 1\} \quad V^* = ? \quad V^+ = ?$



2.5.2 集合的运算及语言定义

2 字母表构成符号串举例

例： $L = \{A, B, \dots, Z, a, b, \dots, z\}$, $D = \{0, 1, \dots, 9\}$

L^*

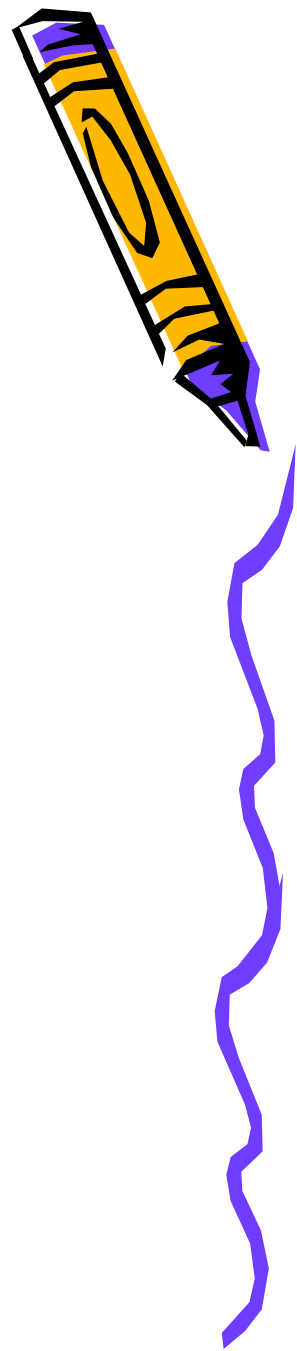
是所有字母串（包括 ϵ ）的集合；

$L(L \cup D)^*$

是以字母开头的字母数字串的集合

LD

是所有一个字母后随一个数字的符号串的集合；



2.5.3 正规式

- 正规式也称为正则表达式，是表示正规集的工具。
- 正规式（regular expression）是说明单词的构成模式的一种重要的表示法，是单词构成规则的描述工具。





■ 正规式和正规集的递归定义：（设字母表为 Σ ）

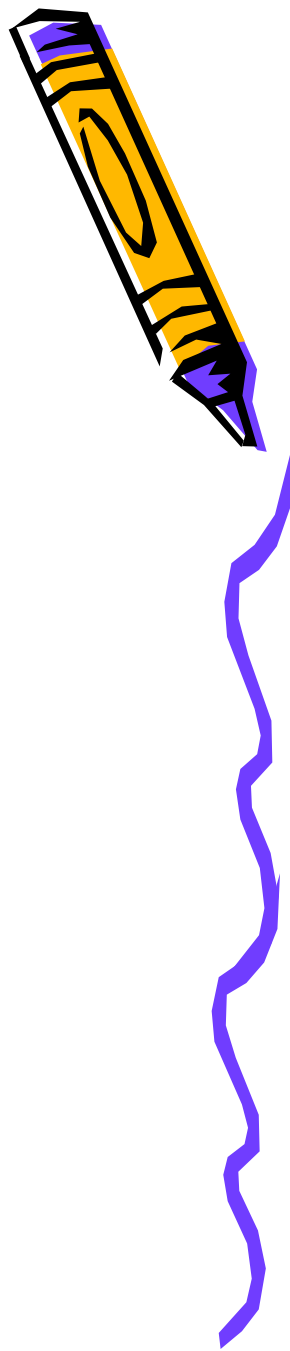
- 1、 ϵ 和 ϕ 都是 Σ 上的正规式，表示的正规集为 $\{\epsilon\}$ 和 $\{\}$ ；
- 2、任何 $a \in \Sigma$ ，则 a 是正规式，表示的正规集为 $\{a\}$ ；
- 3、假定 r 和 s 都是 Σ 上的正规式，表示的正规集为 $L(r)$ 和 $L(s)$ ；
 - ◆ (r) 是正规式，表示的正规集为 $L(r)$
 - ◆ $r|s$ 是正规式，表示 $L(r) \cup L(s)$ ；
 - ◆ $r \cdot s$ 是正规式，表示 $L(r)L(s)$ ；
 - ◆ r^* 是正规式，表示 $(L(r))^*$ ；
- ◆ 4、有限次使用上述三步骤而定义的表达式才是 Σ 上的正规式，仅由这些正规式所表示的集合才是 Σ 上的正规集。



|——或； . ——连接； * ——闭包

规定优先顺序为 “*”、 “.”、 “|”

$(a)|((b)^*(c)) \longrightarrow a|b^*c$



例1: 令 $\Sigma = \{a, b\}$, Σ 上的正规式和相应的正规集有

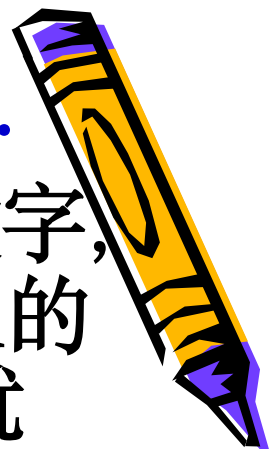
正规式	正规集
a	{a}
ba*	所有以b开头后跟任意多个a的串
a b	{a,b}
ab	{ab}
(a b)(a b)	{aa,ab,ba,bb}
a*	{ϵ, a, aa,} 任意个a的串
(a b)*	{ϵ, a, b, aa, ab} 所有由a 或b组成的串
(a b)*(aa bb)(a b)*	所有含有两个相继的a或两个相继的b的串

程序设计语言的单词都能用正规式来定义.

例2: 令 $\Sigma = \{1, d\}$, 1 代表字母, d 代表数字, 则 Σ 上的正规式: $r = 1(1|d)^*$ 定义的正规集为: $\{1, 11, 1d, 111, 1dd, \dots\}$, 就是 Pascal 和多数程序设计语言允许的标识符的词法规则。

例3: 令 $\Sigma = \{d, ., e, +, -\}$, 其中 d 为 0--9 中的数字。用正规式表示 PASCAL 语言中的无符号实数。比如: 2, 12.59, 3.6e2, 471.88e-1 等都是正规集中的元素。

则 Σ 上的正规式: $d^*(.dd^*|\epsilon)(e(+|-|\epsilon)dd^*|\epsilon)$



练习

1、 $\Sigma = \{a, b\}$ ，则 Σ 上所有以b开头，后跟若干个ab的字的全体所对应的正规式。

$$b(ab)^*$$

2、 $\Sigma = \{a, b\}$ ，写出不以a开头，但以aa结尾的字符串集合的正规式。

$$b(a|b)^*aa$$



- 思考题:

$\Sigma = \{d, .\}$, 则 Σ 上表示**无符号数**的正规式是什么? (不考虑含 e 的科学计数法, 其中 d 为0-9的数字)

如: 2 , 12.59 , 471.88等都是该集合中的元素。

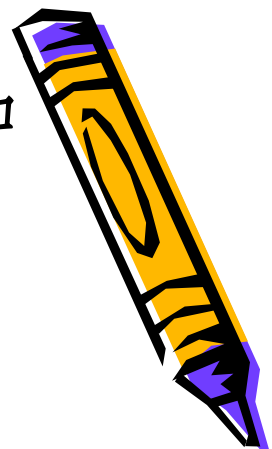
$$dd^*(.dd^* | \varepsilon)$$



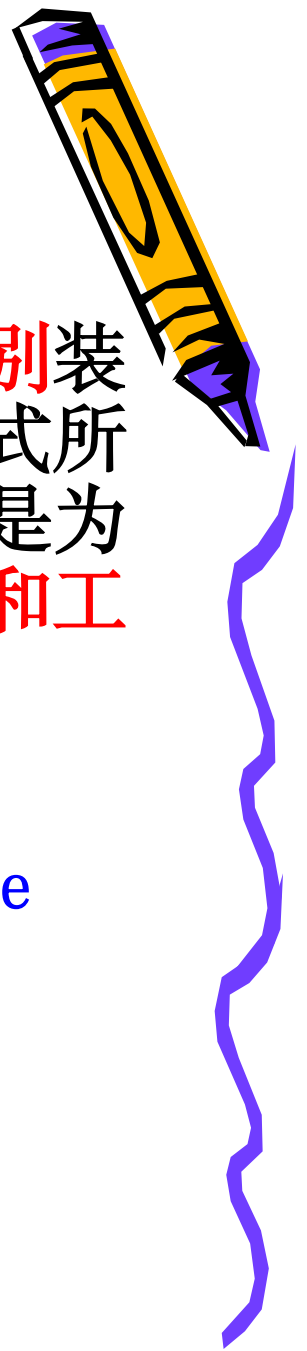
若两个正规式 e_1 和 e_2 所表示的正规集相同, 则 e_1 和 e_2 等价, 写作 $e_1=e_2$ 。

设 r, s, t 为正规式, 正规式服从的代数规律有:

1. $r | s = s | r$ “或”服从交换律
2. $r | (s | t) = (r | s) | t$ “或”的可结合律
3. $(rs)t = r(st)$ “连接”的可结合
4. $r(s | t) = rs | rt$ $(s | t)r = sr | tr$ 分配律
5. $\varepsilon r = r\varepsilon = r$ 是“连接”的恒等元素 零一
6. $e^* = e^+ | \varepsilon$
7. $e^+ = e^*e = ee^*$
8. $(e^*)^* = e^*$



2.5.4 有穷自动机

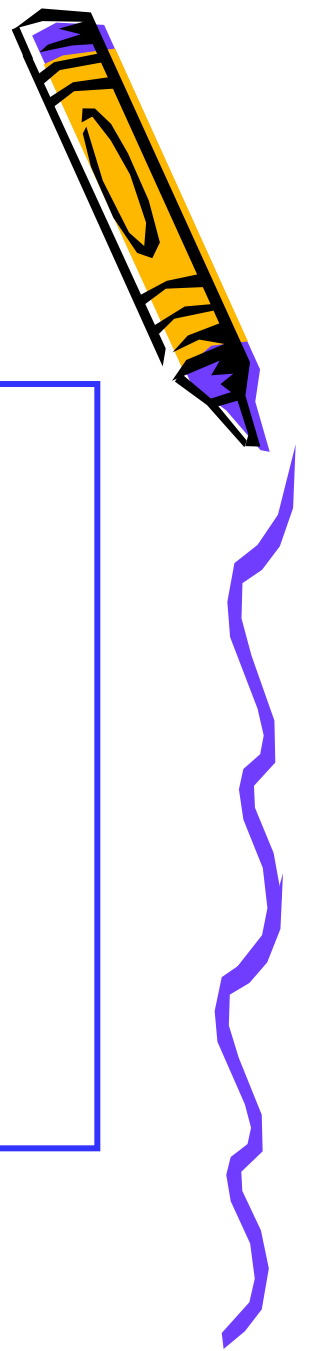


- 有穷自动机(也称有限自动机)作为一种**识别**装置,它能准确地识别正规集,即识别正规式所表示的集合,引入有穷自动机这个理论,是为词法分析程序的**自动构造寻找特殊的方法和工具**。
- 有穷自动机分为两类:
 - 确定的有穷自动机(DFA:Deterministic Finite Automata)
 - 不确定的有穷自动机(NFA:Nondeterministic Finite Automata)



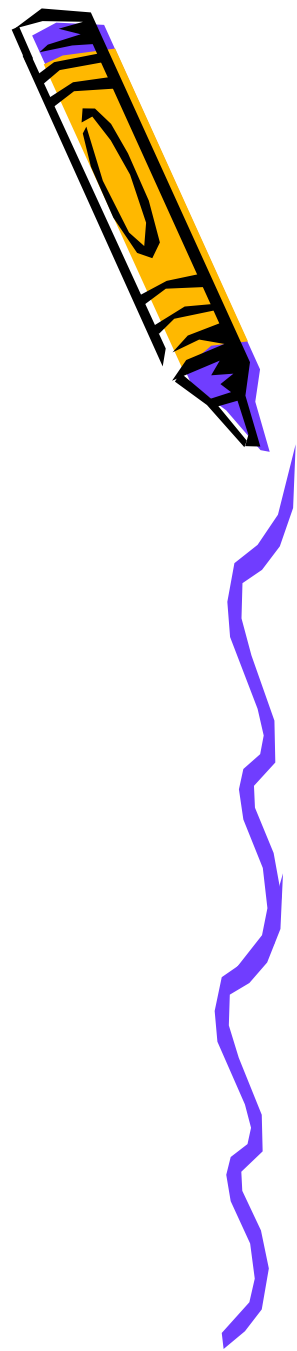
1 不确定的有穷自动机NFA

- 定义：不确定的有穷自动机NFA也是一个五元组， $M = \{S, \Sigma, \delta, s_0, F\}$ ，其中：
 - S 为有穷状态集，
 - Σ 为有穷输入字母表，
 - δ 为 $S \times \Sigma^*$ 到 S 的幂集 (2^S) 的一种映射：
$$S \times \Sigma^* \rightarrow 2^S$$
 - $s_0 \subseteq S$ 是初始状态集，
 - $F \subseteq S$ 为终止状态集 (可空)。



NFA的三种表示法

- 1. 给出NFA的五个部分的具体值
- 2. 矩阵表示法（状态转换表）
- 3. 状态转换图表示法



NFA的矩阵表示

- 例4: NFA $M = (\{S, P, Z\}, \{0, 1\}, \delta, \{S, P\}, \{Z\})$

其中:

$$\delta(S, 0) = \{P\}$$

$$\delta(S, 1) = \{S, Z\}$$

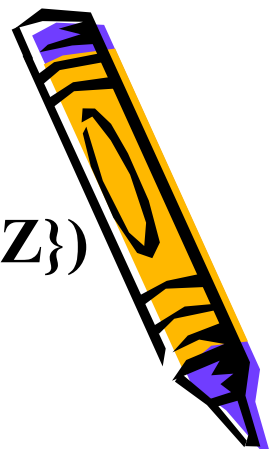
$$\delta(Z, 0) = \{P\}$$

$$\delta(Z, 1) = \{P\}$$

$$\delta(P, 1) = \{Z\}$$

- 矩阵表示

状态 \ 输入	0	1
S	{P}	{S, Z}
P	{ }	{Z}
Z	{P}	{P}



状态图表示

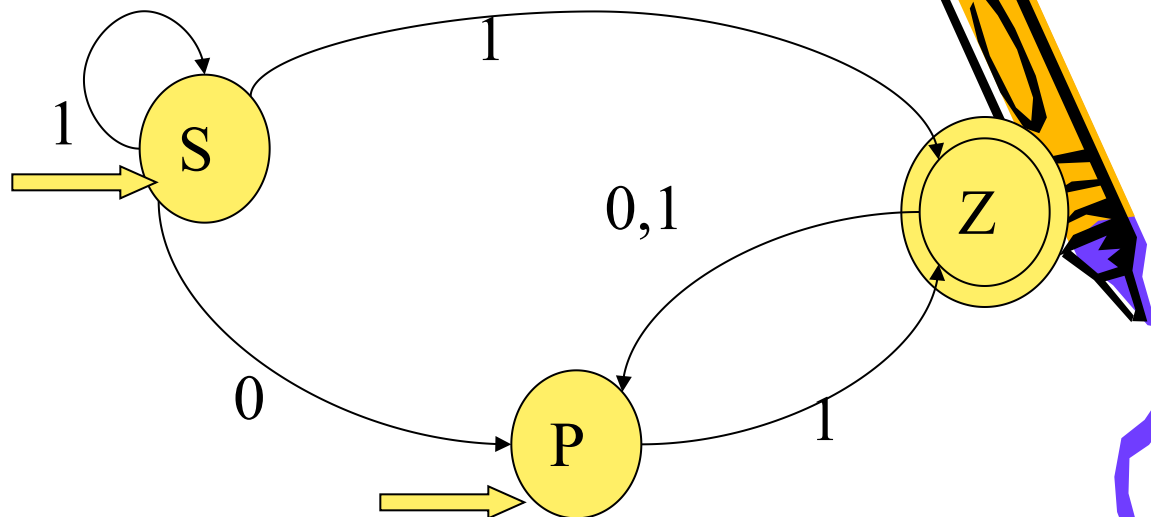
$\delta(S, 0) = \{P\}$

$\delta(S, 1) = \{S, Z\}$

$\delta(Z, 0) = \{P\}$

$\delta(Z, 1) = \{P\}$

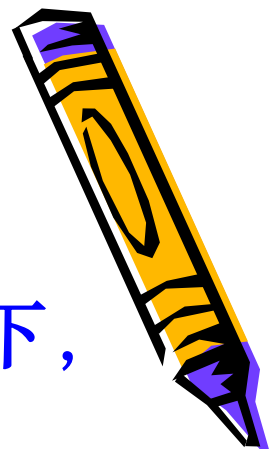
$\delta(P, 1) = \{Z\}$



一个含有 m 个状态， n 个输入字符的NFA的状态转换图：有 m 个结点，每个结点可射出若干条弧与别的结点相连接，每条弧用一个输入符号或 ε 来标记表示(这些字可以相同，也可以是 ε)。

整个图至少有一个初始结点以及若干个(可以是0个)终态结点，某些结点既可以是初态结点，又可以是终态结点。





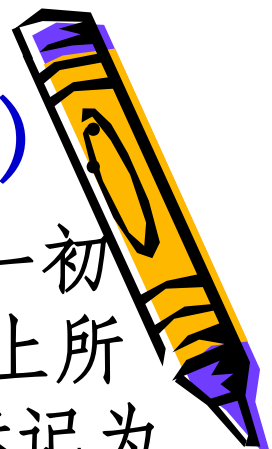
NFA的特点是它的不确定性，即在当前状态下，读入同一个符号，可能有多个下一状态：

- 在NFA的定义中，就是 δ 函数是一对多的；
- 反映在状态转换图中，就是从从一个结点出发，可能有多于一条标记相同的弧转移到不同的下一状态；
- 反映在转换表中，就是 $M[i, a]$ 的值不是一个单一状态，而是一个状态集合。



Σ^* 上的符号串 t 被NFA M 接受（识别）

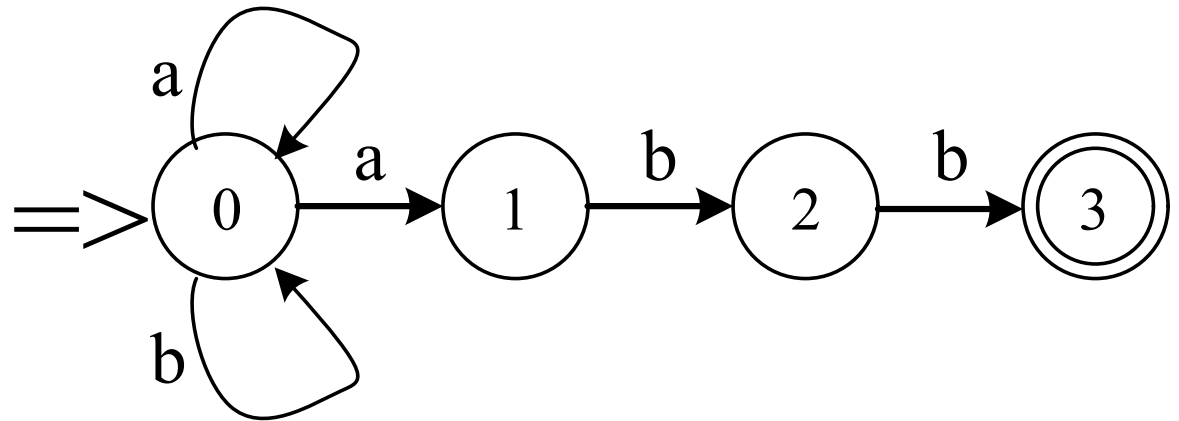
- 对于 Σ^* 中的任何一个串 t ，若存在一条从某一初态结点到某一终态结点的通路，且这条通路上所有弧的标记字依序连接成的串（不理睬那些标记为 ε 的弧）等于 t ，则称 t 可为NFA M 所识别（读出或接受）。
- 若 M 的某些结点既是初态结点又是终态结点；或者存在一条从某个初态结点到某个终态结点的道路，其上所有弧的标记均为 ε ，那么空字 ε 可为 M 所接受。
- NFA M 所能识别的符号串的全体记为 $L(M)$ 。



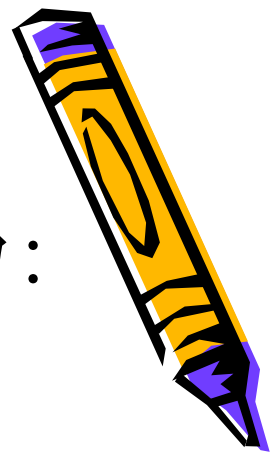
有NFA $M = (\{0,1,2,3\}, \{a,b\}, \delta, 0, \{3\})$, δ 为:

$$\delta(0,a) = \{0,1\} \quad \delta(0,b) = \{0\}$$

$$\delta(1,b) = \{2\} \quad \delta(2,b) = \{3\}$$



该NFA M 所识别的语言为 $L(M) = \{ (a|b)^*abb \}$



2 确定的有穷自动机DFA

一个**确定的有穷自动机** (DFA) M 是一个五元组: $M = (S, \Sigma, \delta, s_0, F)$, 其中:

1. S 是一个**有穷集**, 它的每个元素称为一个状态;
2. Σ 是一个**有穷字母表**, 它的每个元素称为一个输入符号, 所以也称 Σ 为输入符号表;
3. δ 是转换函数, 是在 $S \times \Sigma \rightarrow S$ 上的**单值**映射, $\delta(s, a) = s'$ ($s \in S, s' \in S$), 就意味着, 当前状态为 s , 输入符为 a 时, 将转换为下一个状态 s' , 我们把 s' 称作 s 的一个后继状态;
4. $s_0 \in S$ 是**唯一**的一个初态;
5. $F \subseteq S$ 是一个**终态集 (可空)**, 也称可接受状态或结束状态。

DFA的矩阵表示

例3：有DFA $M = (\{0, 1, 2, 3\}, \{a, b\}, \delta, 0, \{3\})$

δ 为： $\delta(0,a) = 1$ $\delta(0,b) = 2$

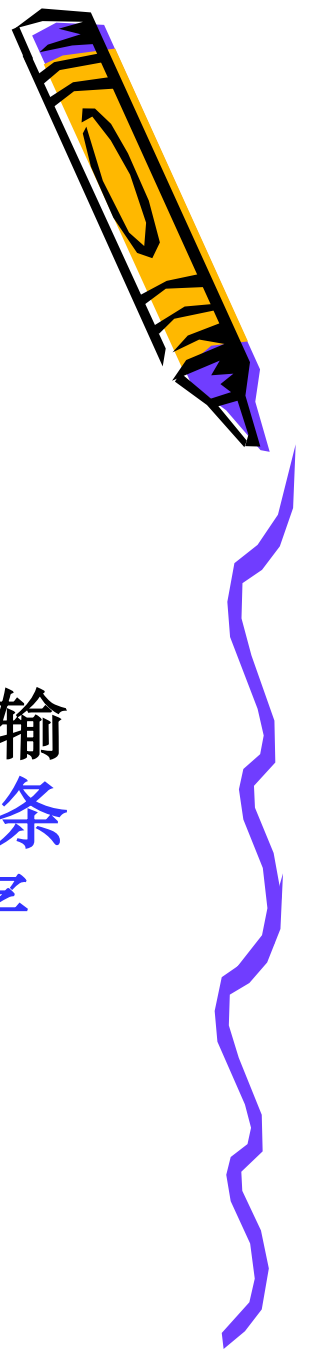
$\delta(1,a) = 3$ $\delta(1,b) = 2$

$\delta(2,a) = 1$ $\delta(2,b) = 3$

$\delta(3,a) = 3$ $\delta(3,b) = 3$

行表示状态，列表示输入字符，矩阵元素表示 $\delta(s, a)$ 的值，称为状态转换矩阵。

状态 \ 输入	a	b
0	1	2
1	3	2
2	1	3
3	3	3



- DFA的确定性表现在：

- 对任何状态 $s \in S$ ，在读入了输入符号 $a \in \Sigma$ 之后，能够**唯一地确定**下一个状态
- 映射函数 $\delta : S \times \Sigma \rightarrow S$ 是一个**单值函数**
- 从状态转换图来看，若字母表 Σ 含有 n 个输入字符，那末任何一个状态结点**最多有 n 条弧射出**，而且每条弧以一个**不同的输入字符**标记。



- 字符串 α 可为DFA M 所接受（识别）：

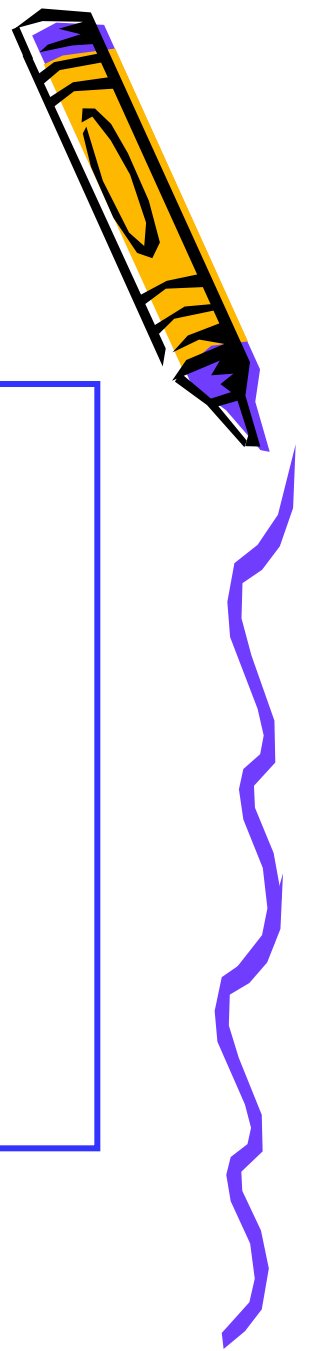
对于 Σ^* 中的任何字符串 α ，若存在一条从初态结点到某个终态结点的通路，且这条通路上所有弧的标记符号连接成的字符串等于 α 。

- 若 M 的初态结点又是终态结点，则空字 ε 可为 M 所识别。
- DFA M 所能识别的符号串的全体记为 $L(M)$ 。
- 对于任何两个有穷自动机 M 和 M' ，如果 $L(M) = L(M')$ ，则称 M 与 M' 是等价的。



小结：不确定的有穷自动机NFA

- 定义：不确定的有穷自动机NFA是一个五元组， $M = \{S, \Sigma, \delta, s_0, F\}$ ，其中：
 - S 为有穷状态集，
 - Σ 为有穷输入字母表，
 - δ 为 $S \times \Sigma^*$ 到 S 的幂集 (2^S) 的一种映射：
$$S \times \Sigma^* \rightarrow 2^S$$
 - $s_0 \in S$ 是初始状态集，
 - $F \subseteq S$ 为终止状态集 (可空)。



小结：确定的有穷自动机DFA

一个**确定的有穷自动机** (DFA) M 是一个五元组： $M = (S, \Sigma, \delta, s_0, F)$ ，其中：

1. S 是一个**有穷集**，它的每个元素称为一个状态；
2. Σ 是一个**有穷字母表**，它的每个元素称为一个输入符号，所以也称 Σ 为输入符号表；
3. δ 是转换函数，是在 $S \times \Sigma \rightarrow S$ 上的**单值**映射， $\delta(s, a) = s'$ ($s \in S, s' \in S$)，就意味着，当前状态为 s ，输入符为 a 时，将转换为下一个状态 s' ，我们把 s' 称作 s 的一个后继状态；
4. $s_0 \in S$ 是**唯一**的一个初态；
5. $F \subseteq S$ 是一个**终态集 (可空)**，也称可接受状态或结束状态。

小结



- Σ^* 上的符号串 t 被 NFA M 接受 (识别)

对于 Σ^* 中的任何一个串 t ，若存在一条从某一初态结点到某一终态结点的通路，且这条通路上所有弧的标记字依序连接成的串 (不理睬那些标记为 ε 的弧) 等于 t ，则称 t 可为 NFA M 所识别 (读出或接受)。NFA M 所能识别的符号串的全体记为 $L(M)$ 。

- Σ^* 上的符号串 t 被 DFA M 所接受 (识别)：

对于 Σ^* 中的任何字符串 α ，若存在一条从初态结点到某个终态结点的通路，且这条通路上所有弧的标记符号连接成的字符串等于 α 。DFA M 所能识别的符号串的全体记为 $L(M)$ 。



DFA与NFA的主要区别

- (1) DFA任何状态都没有 ε 转换，即没有任何状态可以不进行输入符号的匹配就直接进入下一个状态；
- (2) DFA对任何状态 s 和任何输入符号 a ，最多只有一条标记为 a 的边离开 s ，即转换函数 $\delta : S \times \Sigma \rightarrow S$ 是一个单值部分函数；
- (3) DFA的初态唯一，NFA的初态为一集合。

DFA是NFA吗？

